

Software architecture

Software architecture

Rodrigo Martins Pagliares

Microservices architecture

Monolithic vs. microservices architecture

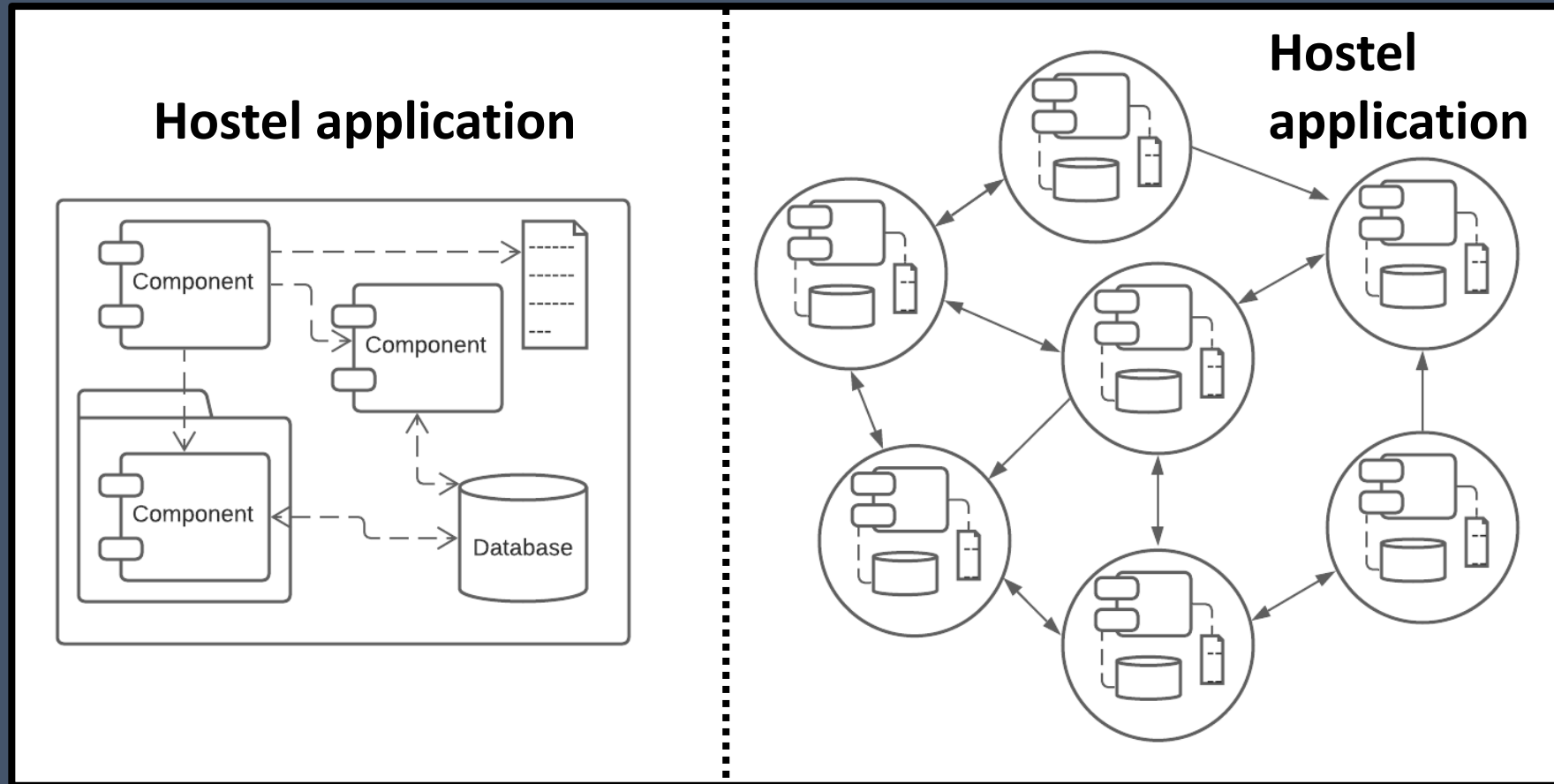
Lecture: Monolithic vs. microservices architecture

Module: Microservices architecture

Monolithic vs. microservice architecture

- **Monolithic architecture:**
 - ✓ Self contained with all parts of the application packaged together.
- **Microservices architecture:**
 - ✓ The idea is to design a large distributive scalable application.
 - ✓ The application consists of cohesive services.
 - ✓ The services are available to evolve or even be completely re-written over the life of the application.

Monolithic vs. microservice architecture



Monolithic architecture

Lecture: Monolithic architecture

Module: Microservices architecture

Monolithic application

- Self-contained application in which all parts are packaged together.
- In three-tiered application architecture, this means that the Presentation, Service, and Persistence layers are packaged and deployed together.



Source: <https://bit.ly/3uCY5oP>

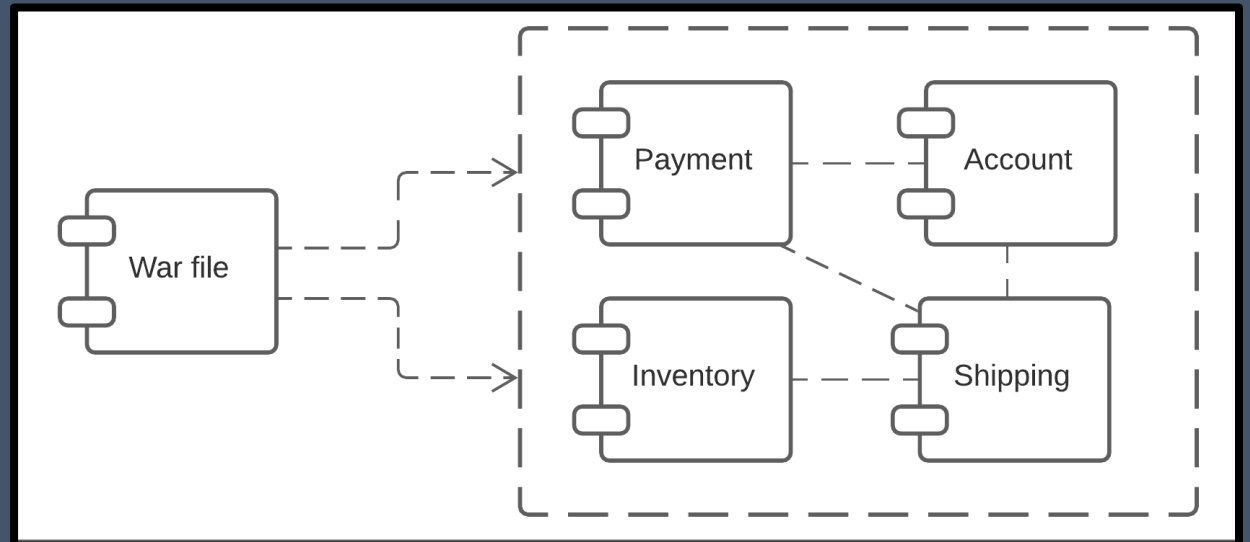
Monolithic application

- The most common way to package and deploy to an application server a monolithic web application is to include all resources, components, and class files into a single file.
 - ✓ WAR file in Jakarta EE, for example.



Example of a monolithic application

- An e-commerce application includes components that process payments, control inventory, manage shipping, manage customer's accounts, generate front-end views, etc.
- All components are included in one monolithic application and packaged and deployed to a web/application server.
 - WAR, EAR



Load balancers and monolithic applications

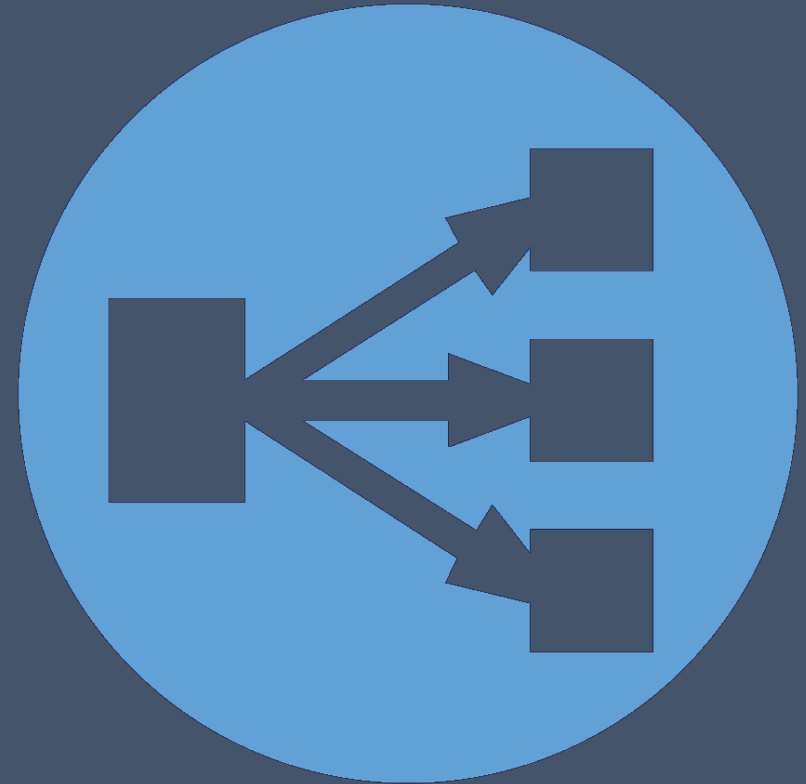
- In the early days of the Internet, web applications were **monolithic**.
 - ✓ A huge demand of visitors slows down the application and possibly stop it all together.
- The **problem** was that the **resources** could not be provided quickly enough to meet demand.



Source: <https://bit.ly/3bIXZ6b>

Load balancers and monolithic applications

- To cope with the scalability and availability problems, we can run several instances of the application behind a load balancer.



Source: <https://bit.ly/3dYaWvw>

Advantages of monolithic architectures

Advantages of monolythic architectures

- Simple way to develop and deploy an application.
 - ✓ There's only **one application** to test, to deploy, and to worry about.
- IDEs are designed with the **monolithic architecture** in mind.
 - ✓ Optimized for the **development** and **deployment**.



Disadvantages of monolithic architectures

Disdvantages of monolythic architectures

- Applications are often very large.
 - ✓ Difficult for developers to understand and maintain.
 - ✓ Especially for developers who have recently joined the team.
- ✓ It is time-consuming to thoroughly understand an application.



Disdvantages of monolythic architectures

- Frequent deployments are not practical.
 - ✓ Require the coordination of many developers and departments.
 - ✓ It may take hours or days to arrange a deployment, hindering the testing of new features and bug fixes.
- ✓ It does not scale well.



Disdvantages of monolythic architectures

- It's **difficult** to **change** the **technology** or **framework** it **uses**.
 - ✓ The **application** was **developed** based on **technology** **decisions** made at the **beginning** of the **project** and you are **locked** with these **decisions**.
- If a **technology** is **developed** that **solves** a **problem** in a **better** and **more performant way**, it is really **difficult** and **expensive** to **refactor** and **start using** the **new technology**.
 - ✓ **Rewriting** an **entire application** is **almost never** an **affordable option**.

Final word about monolithic architectures

- **Regardless of its drawbacks, the monolithic architecture has supported large organizations for a long time.**
- **Decades of experience deploying monolithic applications helped developing effective ways of dealing with the shortcomings.**

Final word about monolithic architectures

- The challenges of nowadays complex applications demand alternate solutions to monolithic applications.
 - ✓ Among those challenges is the unpredictable nature of the user's demand on the applications available on the Internet.
- Microservices is one possible solution.

What are microservices?

Lecture: What are microservices?

Module: Microservices architecture

What are microservices?

- A.K.A. **microsevice**s architecture.
- **Architectural style** that **structures** an **application** as a **collection** of **small services** that are:
 - ✓ **Highly maintainable** and **testable**
 - ✓ **Loosely coupled**
 - ✓ **Independently deployable**
 - ✓ **Organized** around **business** capabilities
 - ✓ **Owned** by a **small team**.
- Each **microservice** runs in its **own process** and **typically communicates** via **RESTful**.

The objective of microservices

- Decompose a large monolithic application into much smaller applications to solve development and evolutionary problems.



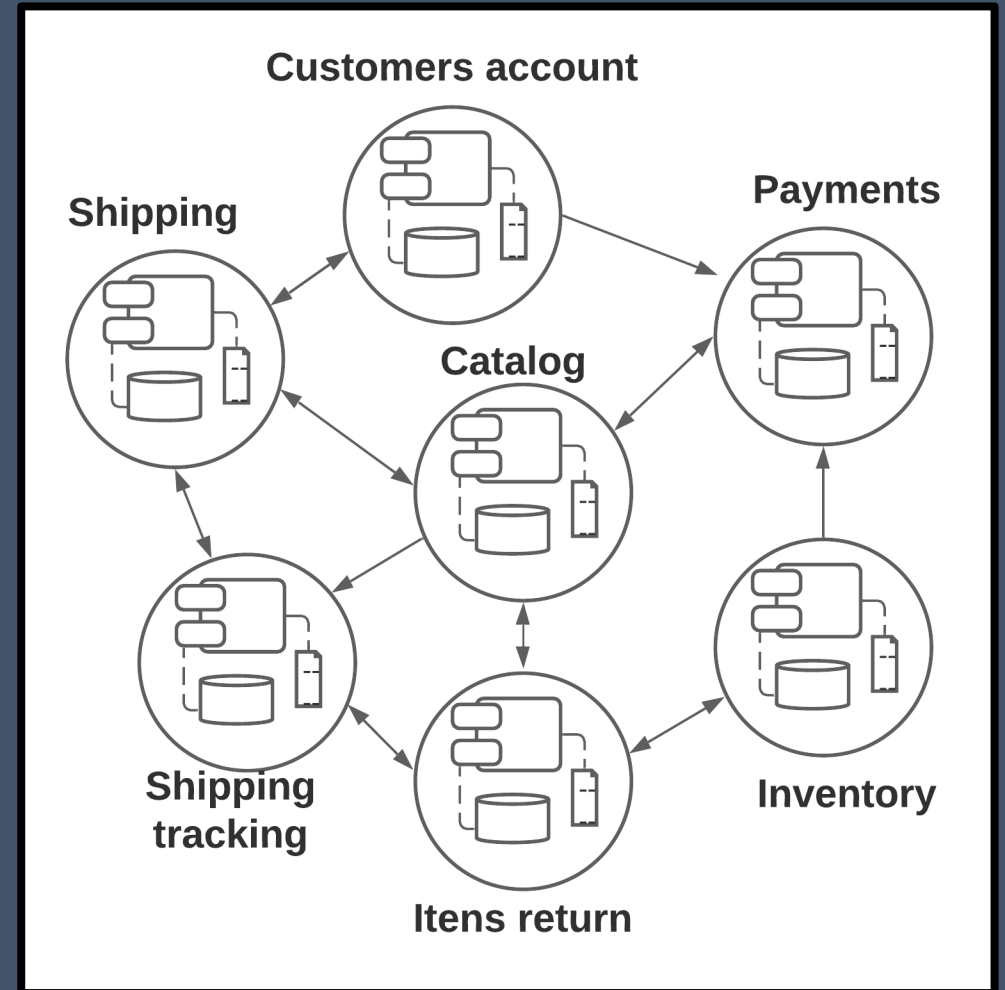
Source: <https://bit.ly/2NXJkMe>

Microservice architecture

- Has become a **hotly discussed** and popular **pattern** over the **past few years**.
- The **idea** is to **design** a **large distributed scalable application** that **consists** of **small cohesive services** that are **able** to **evolve** or **even** be **completely rewritten** over the **life** of the **application**.
 - ✓ This is **not** a **new idea**.
 - ✓ It is **similar** to the **servers oriented architecture pattern** that has been used for a **long time**.

Microservices architecture

- Each service should be small and handle only one business concern.
 - ✓ A microservice.



Advantages and disadvantages of microservices

Lecture: Advantages and disadvantages of microservices

Module: Microservices architecture

Advantages of microservices

Advantages of microservices

ID	Advantage	ID	Advantage
01	Microservices are small.	07	Scalability is built into the microservice architecture.
02	Microservices are easy to understand.	08	The user interface is normally divorced from back-end development .
03	It is easy to change the technology stack when using microservices .	09	It is possible to deploy each microservice onto its own tailored environment .
04	Microservices are deployed quicker and often.	10	Resource allocations to microservice can be fine-grained.
05	Local changes to a microservice code do not affect other microservices .	11	Fault tolerance and isolation are increased when using microservices .
06	Microservices can be developed and deployed by independent teams.		

Advantages of microservices architecture

- **Microservices are small and therefore enable quick development.**
- **Easy to understand.**
- **Easy to change the technology stack.**
 - ✓ Because each **microservice** is **small**, **rewriting** is **not expensive**.
 - ✓ New **microservices** can be **rewritten** in any **programming language**



Source: <https://bit.ly/2Mus40K>

Advantages of microservices architecture

- Each **microservice application** can be **deployed quicker and often**.
 - ✓ Without the **cooperation** of other teams because each service stands **alone**.
- **Local changes to code DO NOT** affect other **microservices**.
 - ✓ **Continuous development** is **possible**.



Source: <https://bit.ly/3uCP3bf>

Advantages of microservices architecture

- Each **microservice** can be **developed** and **deployed** by **developers** independently of other **development teams**.
- **Scalability** is built into the **architecture**.



Source: <https://bit.ly/3dOljCI>

Advantages of microservices architecture

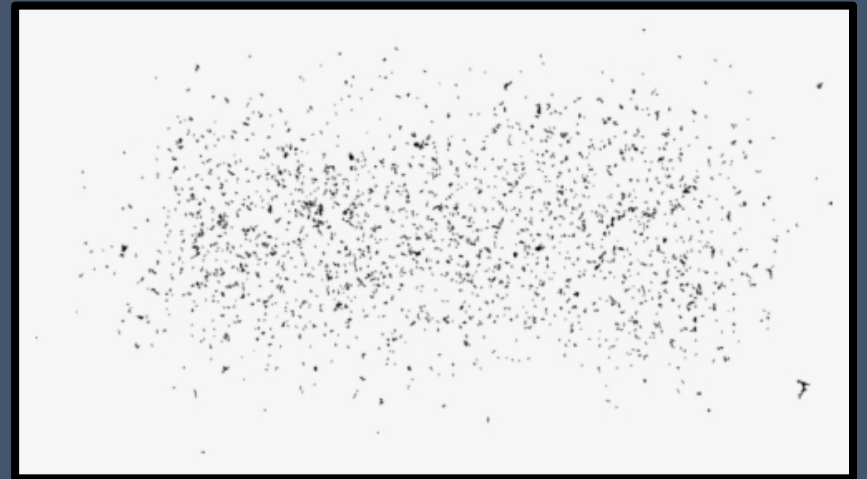
- The **user interface** is normally **divorced** from **back-end development**.
 - ✓ Your team may never even see a line of UI code.
 - ✓ Separation of concerns.
- **Example:**
 - ✓ If you're **programming** to a **REST API**, you're only **required** to **honor** the **resource representation contract**
 - ✓ **No need** to think about the way the **front end** will be **implemented**.



Source: <https://bit.ly/3q1ulOP>

Advantages of microservices architecture

- Able to **deploy** each **microservice** onto its own **tailored environment**.
- The **resource requirements** for your **microservices** may **differ** from another, thereby allowing **resource allocations** to be **fine-grained**.



Advantages of microservices architecture

- **Fault tolerance and isolation are increased.**
 - ✓ A fault in **one microservice** does not affect the **operations** of the **others**, allowing the **application** to **continue to perform**.
- **Example:**
 - ✓ In a system that uses **message orientating middleware** for **communication** between **services**, **messages** destined for a **failed microservice** wait in the **queue** until the **fault** is **resolved**.



Source: <https://bit.ly/2MzJ45U>

Disadvantages of microservices

Disadvantages of microservices

ID	Disadvantage	ID	Disadvantage
01	The task of maintaining and orchestrating microservices increases as the number of microservices grows.	05	Large application that consists of hundreds of modules can take considerable time to load.
02	Cross-cutting changes mean that all microservices must update their code to keep in sync.	06	Communicating via networks that suffer from several problems.
03	The risk of duplicated code is high when using microservices.	07	Constant monitoring of network performance is vital.
04	Global testing is difficult.	08	Debugging problems can be harder.

The benefits of the microservice architecture do not come for free. There are a few costs involved.

Disadvantages of microservices architecture

- The ease with which you can **develop** a **microservice** makes it **easy** for the **number of microservices** to **grow**.
 - ✓ **100 microservices** can easily become **1,000** or **more**.
- This **poses several difficulties** related to **orchestration** and **management**.



Source: <https://bit.ly/3aX8Ry6>

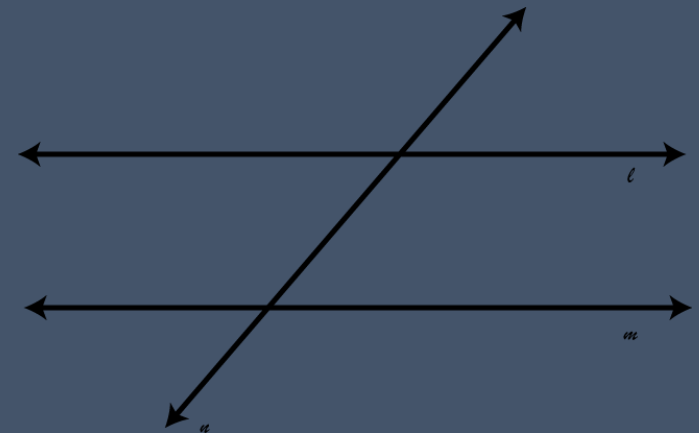
Disadvantages of microservices architecture

- **Maintaining a handful of services is not a difficult task.**
 - ✓ However, as the number of microservices grows, the task of maintaining them increases.
- **A significant investment needs to be made to ensure that the microservices are deployed and maintained.**
 - ✓ Processes need to be automated to reduce the burden of deploying and maintaining a large number of services.



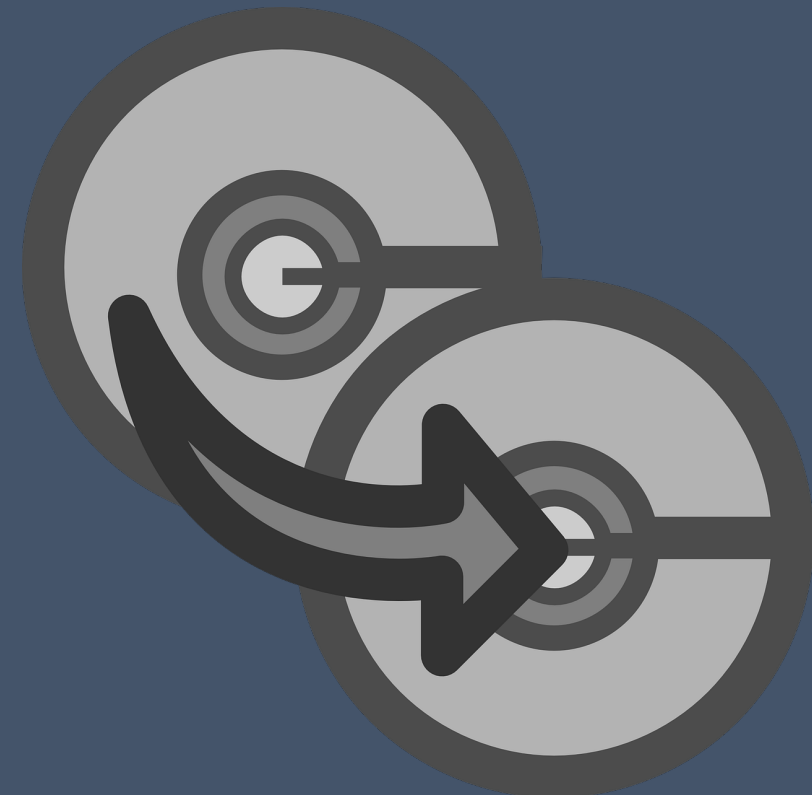
Disadvantages of microservices architecture

- **Cross-cutting changes** mean that all **microservices** must **update their code to keep in sync**.
 - ✓ It can be **time consuming to perform** and has **significant retesting costs**.
- **Alterations to interface contracts and message formats** can be the **cause of such changes**
 - ✓ **Requiring that all teams work in coordinated ways**.
 - ✓ **Higher costs as the number of microservices grows**.



Disadvantages of microservices architecture

- In a **microservice architecture**, the risk of **duplicated code** is high.
- To avoid **coupling** and **dependencies**, code must sometimes be **duplicated**.
 - ✓ **Meaning** every instance of that **code** must be **tested** and **maintained**.



Disadvantages of microservices architecture

- A large application that consists of hundreds of modules can take considerable time to load.
 - ✓ Negatively effects the developers' productivity.



Source: <https://bit.ly/37RhY1o>

Disadvantages of microservices architecture

- **Global testing is difficult.**
 - ✓ In a **monolithic approach**, we would just need to launch our **WAR** on an **application server** and ensure its **connectivity** with the **underlying database**.
 - ✓ With **microservices**, each **dependent service** needs to be **confirmed** before **testing** can occur.



Source: <https://bit.ly/3q0ZkKS>

Disadvantages of microservices architecture

- **Every service may be hosted in a distributed way, communicating via networks that suffer from:**
 - ✓ **latency** issues
 - ✓ **incompatible** versions
 - ✓ **unreliable** providers
 - ✓ **hardware** problems
 - ✓ **and more.**
- **Constant monitoring of network performance is vital.**



Disadvantages of microservices architecture

- **Debugging problems can be harder.**
 - ✓ Each service has its own set of logs to go through.
 - ✓ Log, logs, and more logs.



Questions?